

Kernel Boot und Initialisierung

Vom Firmware-Reset bis zum ersten Prozess im User Space

5.02

Drei Fragen strukturieren den Bootpfad

01 Wer lädt den Kernel?

Firmware und Bootloader bereiten Image, Parameter und Hardwarebeschreibung vor.

02 Was tut der Kernel?

`start_kernel()`, Initcalls und Treiber werden geordnet aktiviert.

03 Wie startet PID 1?

Initramfs und Root-Dateisystem führen zum ersten User-Space-Prozess.

Lernziel

Am Ende kannst du den Bootpfad einer Linux-Plattform technisch nachvollziehen.

Vier Phasen verbinden Reset, Kernel und User Space

Firmware

Startet die CPU und wählt das Bootmedium.

Bootloader

Lädt Kernel, initramfs und Hardwarebeschreibung (DTB).

Kernel

Initialisiert Architektur, Speicher, Scheduler und Treiber.

PID 1

Übernimmt Dienste und den weiteren Systemstart.

BOOTVERTRAG

Übergabe: Kernel-Image, Bootparameter und Hardwarebeschreibung

Kernelaufgabe: Ressourcen initialisieren und Geräte binden

Abschluss: Root-Dateisystem bereitstellen und PID 1 starten

Firmware und Bootloader bereiten den Kernelstart vor

BOOTKETTE

Reset

Firmware

Bootauswahl

Bootloader

Kernel-Image + initramfs + DTB

Übergabepunkte

01 Firmware

Initialisiert die für den Start benötigte Plattform.

02 Bootloader

Lädt Artefakte an vereinbarte Speicheradressen.

03 Vertrag

Übergibt Bootparameter und Hardwarebeschreibung.

Architekturspezifisch beginnt der Kernelstart

QUELLPFADE

```
arch/x86/boot/  
arch/x86/boot/compressed/  
arch/x86/kernel/head_64.S  
init/main.c
```

Der frühe Pfad

01 Startzustand

Register, CPU-Modus und Übergabeparameter prüfen

02 Dekompression

Kernel-Image entpacken und gegebenenfalls verlagern

03 Architekturstart

Seitentabellen, Stack und C-Laufzeit vorbereiten

Frühe Initialisierung schafft Arbeitsfähigkeit

QUELLCODE

```
setup_arch(&command_line);  
mm_init();  
sched_init();  
init_IRQ();  
time_init();
```

Zwei Ebenen

Früher Einstieg

CPU-Zustand, Stack und Seitentabellen vorbereiten

Im generischen Kernel

Speicher, Scheduler, Interrupts und Zeitbasis werden schrittweise nutzbar

Frühe Initialisierung darf keine späteren Kernel-Dienste voraussetzen.

start_kernel() ordnet die generische Initialisierung

INIT/MAIN.C

```
asmlinkage __visible void  
start_kernel(void)  
{  
    ...  
}
```

Lies als Reihenfolge

01 Grundlagen

Command Line, CPU, Speicher und Scheduler

02 Ausführung

Interrupts, Timer, RCU und Workqueues

03 Übergang

rest_init() erzeugt die ersten Kernel-Tasks

Reihenfolge und verfügbare Funktionen sind Teil des Bootvertrags.

Initcalls aktivieren Subsysteme stufenweise

INITCALLS

```
early_initcall(fn)
core_initcall(fn)
subsys_initcall(fn)
fs_initcall(fn)
device_initcall(fn)
late_initcall(fn)
```

Drei Regeln

01

Reihenfolge

Level ordnen Abhängigkeiten grob.

02

Linker-Sektionen

Makros legen Funktionszeiger in definierte Bereiche.

03

Built-in und Module

Built-ins laufen beim Boot, Module beim Laden.

Hardwarebeschreibung führt zur Gerätepopulation

QUELLEN

Device Tree: DTB

ACPI: Tabellen

Firmware Node

Platform Device

Driver Core

Vom Knoten zum Gerät

01

Beschreibung

Firmwaredaten beschreiben Hardware und Ressourcen

02

Population

Busregeln erzeugen passende Geräteobjekte

03

Binding

Treiber und Gerät treffen über Bus-Matching zusammen

Initramfs führt zum Root-Dateisystem

BOOTPARAMETER

```
root=  
rootfstype=  
rootflags=  
rdinit=  
init=
```

Zwei Root-Phasen

Initramfs-Phase

Werkzeuge und Treiber bereiten die Root-Suche vor.

Endgültiges Root

Nach Mount und Wechsel übernimmt das reale Root-Dateisystem.

Fehler bei Treiber, Gerät oder Parameter enden oft in früher Shell oder Kernel Panic.

PID 1 schließt die Kernelinitialisierung ab

INIT/MAIN.C

```
kernel_init()  
prepare_namespace()  
run_init_process()  
/sbin/init
```

Der Übergang

01

Kernel-Task

kernel_init wartet auf notwendige Initialisierung.

02

Root bereit

Namespace und Root-Dateisystem werden vorbereitet.

03

User Space

PID 1 startet Dienste und verwaltet Prozesse.

Der Kernel bleibt aktiv; die Systempolitik liegt nun im User Space.

Bootparameter verändern frühe Entscheidungen

`/PROC/CMDLINE`

```
console=  
root=  
ro oder rw  
init=  
loglevel=  
quiet
```

Drei Klassen

01

Ausgabe

Konsole und Log-Level steuern frühe Diagnose.

02

Plattform

Root-Gerät, Treiberoptionen und Speichergrenzen.

03

Fehleranalyse

Parameter gezielt ändern und Wirkung dokumentieren.

Praxis: Rekonstruiere den Bootpfad

TERMINAL

```
$ cat /proc/cmdline  
$ journalctl -b -k  
$ systemd-analyze time  
$ ps -p 1 -o pid,comm,args
```

Belege drei Aussagen

01

Kernel und Parameter

Release, Kommandozeile und Architektur festhalten

02

Zeit und Treiber

Frühe Logs, Gerätebindung und Verzögerungen finden

03

PID 1

Init-System, Root-Dateisystem und Startzustand erklären

Praxis: Bootcode und Laufzeit verbinden

01 Einstieg finden

Architekturspezifischen Einstiegspfad im Source Tree lokalisieren.

02 `start_kernel()` lesen

Fünf zentrale Initialisierungsschritte auswählen.

03 Initcall zuordnen

Ein `device_initcall()` finden und seinem Subsystem zuordnen.

04 Bootparameter prüfen

Zwei Parameter aus `/proc/cmdline` im Quellcode verfolgen.

05 Laufzeit belegen

Bootlog, Root-Dateisystem und PID 1 dokumentieren.

06 Befund erklären

Quellcodepfad und Laufzeitbeobachtung miteinander verbinden.

Arbeite lesend und nutze die festgelegte Kernelversion des Trainings.

Der Bootpfad verbindet Architektur, Subsysteme und User Space.

Formuliere in einem Satz:

Welche Übergabe entscheidet auf deiner Plattform über den nächsten Bootschrift?